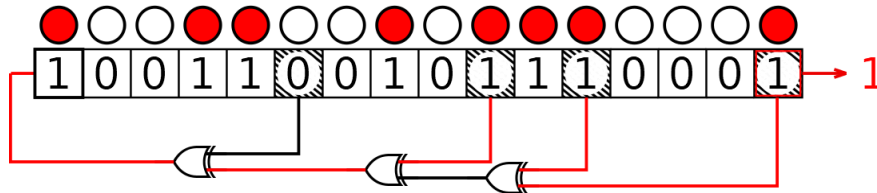


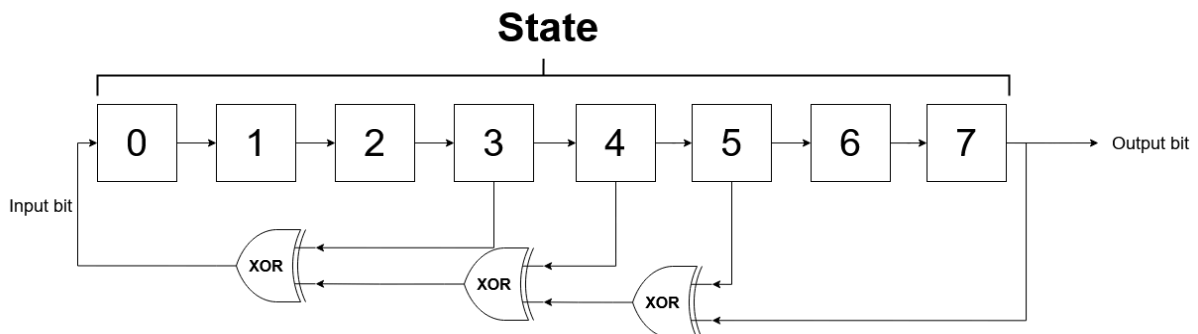
- Zipped **project** folder for Exercise 1.a and 1.b (they should be in the same project)

Exercise 1.a) Implement an LFSR



A Linear Feedback Shift Register (LFSR) is a shift register¹ commonly used to implement Pseudo-Random Number Generators (PRNGs) in hardware. In LFSRs, the input bit is a linear function of its previous content, usually a XOR between some of the bits of the register.

- The content of the LFSR is called “*state*”.
- The initial state is called “*seed*”.
- The next input bit is computed by XORing some of the bits of the current state, called “*taps*”.
- The next state is computed by shifting the register right by one position and inserting the input bit in the leftmost position. For example, if the taps are bits 3, 4, 5, and 7, the LFSR can be graphically represented as below:



At each clock cycle:

- The input bit is computed using the current value of the taps.
- The state is shifted right by one position, outputting the rightmost bit.
- The input bit becomes the new leftmost bit.

Write a program in **C** that implements an 8-bit LFSR on the LANDTIGER board:

- Initialize the state (an unsigned char) with a seed that **you can choose freely** and show it using the LEDs.
Note that choosing a seed equal to 0 will result in the LFSR always outputting a 0.
- Each time KEY1 is pressed, call an **assembly** function called `lfsr_step` that receives the current state and the bit position of the taps and returns the next state. The output bit is stored in an integer variable passed by reference called `output_bit`. The taps are

¹ A shift register is a register which accepts one input bit at a time. The input bit is shifted in from the left, while the rightmost bit is shifted out from the right

represented by an 8-bit variable, where a 1 is present in positions corresponding to the tap positions and 0 elsewhere. For example, for the LFSR above, taps contain the value $0x1D_{16} = 00011101_2$

- After `lfsr_step` returns, update the current state and show it on the LEDs.
The function prototype is:

```
unsigned char lfsr_step(unsigned char current_state, unsigned char taps, int *output_bit);
```

The function arguments are:

Type	Name	Description
unsigned char	current_state	The current 8-bit value stored in the LFSR
unsigned char	taps	An 8-bit mask where each bit set to 1 indicates a tap position. For example, if bit 3 is a tap, then bit 3 of taps must be 1
int *	output_bit	Pointer to an integer where the function will store the output bit (the old least-significant bit).

An example of how `lfsr_step` could look like in **C** when taps are in positions 3, 4, 5 and 7 is given below:

```
// Output bit = old least-significant bit
*output_bit = current_state & 0x01;

// Example taps: positions 3, 4, 5, 7
// Tap at position 3 → shift right by 3
// Tap at position 4 → shift right by 4
// Tap at position 5 → shift right by 5
// Tap at position 7 → shift right by 7

// Compute the feedback bit = XOR of tap bits (7,5,4,3)
unsigned char feedback =
    ((current_state >> 3) ^
     (current_state >> 4) ^
     (current_state >> 5) ^
     (current_state >> 7)) & 0x01;

// Compute the next state: shift left, drop MSB, insert feedback at LSB
unsigned char new_state =
    ((state << 1) & 0xFE) | feedback;
```

For example, if the initial state is $0xAA$ (10101010 in binary), the next 8 states of the LFSR will be:

1. $11010101_2 = 0xD5_{16} = 213_{10}$
2. $11101010_2 = 0xEA_{16} = 234_{10}$
3. $11110101_2 = 0xF5_{16} = 245_{10}$
4. $11111010_2 = 0xFA_{16} = 250_{10}$
5. $01111101_2 = 0x7D_{16} = 125_{10}$

Since the LFSR has a finite number of possible states, after a certain number of iterations, it will eventually return to its initial state and repeat the same sequence of values.

An m -bit LFSR is said to be **maximal-length** if the number of distinct states before it repeats is

exactly $2^m - 1$.

(Note that the all-zero state is excluded, because an LFSR that enters the zero state will remain stuck there forever.)

The length of the sequence depends on both the **number** and the **positions** of the taps.

For example, the LFSR used in **Exercise 1.a** is maximal-length: $2^8 - 1 = 255$.

Q: Did you notice some strange behaviour? If so can you make hypothesis on why it's happening? Il comando dato dalla pressione del tasto a volte puo' ripetersi piu di una volta anche se viene eseguita una sola pressione a del bouncing del bottone.

Exercise 1.b) Experiment with Different Tap Positions

Extend the program from Exercise 1.a by experimenting with different tap positions and implementing the following feature:

You will configure the tap mask directly using the keys and LEDs, and then measure the sequence length produced by the LFSR.

At the beginning, the tap mask is initialized to 0x00 and all LEDs are off. The LEDs always display the current tap mask (LED0 = bit 0, LED7 = bit 7).

Use the keys as follows:

- **KEY0 (INT0) – Toggle bit 0 of the tap mask**
 - Pressing KEY0 toggles the least significant bit (bit 0) of the tap mask.
 - The new tap mask is immediately shown on the LEDs.
- **KEY1 – Shift the tap mask to the next bit**
 - Pressing KEY1 shifts the entire tap mask by one position (e.g., to the left). This “moves” the bits you have already set to the next position.
 - After the shift, you can again use KEY0 to toggle bit 0 and add or remove a tap there.
 - By repeatedly pressing KEY1 (shift) and KEY0 (toggle), you can build any 8-bit tap pattern, which is always visible on the LEDs.
- **KEY2 – Measure the sequence length for the current taps**

When the desired tap configuration is visible on the LEDs:

- Press KEY2 to:
 1. Store the current LED pattern as the tap mask.
 2. Use a fixed non-zero seed (the same as in Exercise 1.a).
 3. Repeatedly call `lfsr_step` in a C loop until the LFSR state returns to the seed or reaches the all-zero state.
 4. Count how many steps were executed (the **length of the sequence**).
 5. Display the sequence length on the LEDs

After the result is displayed, pressing KEY0 resets the tap mask to 0x00, turns all LEDs off, and returns to edit mode, allowing you to create and test a new tap configuration. You must test several different tap configurations and fill the table.

Include at least: one non-maximal tap set, one that reaches the all-zero state, and two maximal-length (primitive) tap sets.

Tap positions	Length of the sequence	Maximal-length? (Yes/No)	Reaches all-zero?
e.g. 3, 4, 5, 7	255	No	Yes
1,3,4,7	00111111	no	
0 1 2 3 4 5 6 7	00001001	no	
0 7	00111111	no	
34	//	yes	
4567	11011001	no	